

LAPORAN PRAKTIKUM

Internet of Think II

Modul 3: Jaringan Nirkabel dan Komunikasi HTTP



Oleh :

Nama : Djaky Fila Hanifan

NPM : 25782067

Kelas : TRI (3C)

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG TAHUN

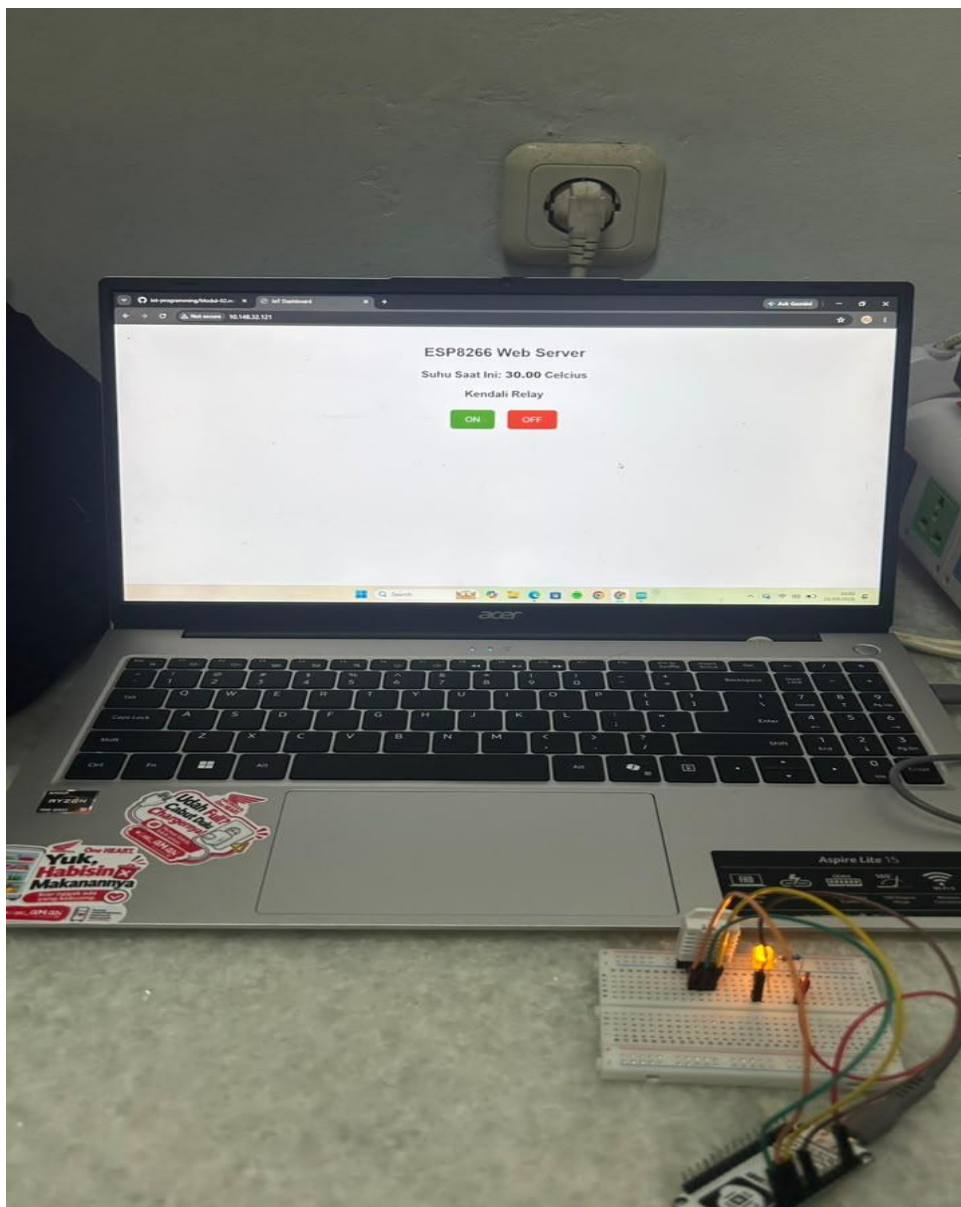
2026

Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

Skenario: Anda merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menghidupkan/mematikan lampu LED.

Langkah Kerja Rangkaian:

1. Hubungkan pin **DATA / OUT / S DHT** ke pin **D4 (GPIO 2)**. (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU. *(Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).*



Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*). Tuliskan kode program di bawah. **Jangan lupa mengganti variabel ssid dan password!**
2. *Catatan teknis*: Kita menggunakan teknik **Raw String Literal C++** (`R"rawliteral(...)rawliteral"`) untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip `"` di setiap baris HTML.
3. **Upload** kode tersebut. Buka **Serial Monitor**, dan catat alamat IP yang didapat (contoh: 192.168.43.15).
4. Hubungkan Laptop/HP Anda ke Wi-Fi yang sama, buka *browser* (Chrome/Safari), lalu ketikkan alamat IP tersebut. Klik tombol ON dan OFF di layar untuk melihat LED menyala dan padam.

Penjelasan Singkat Kode:

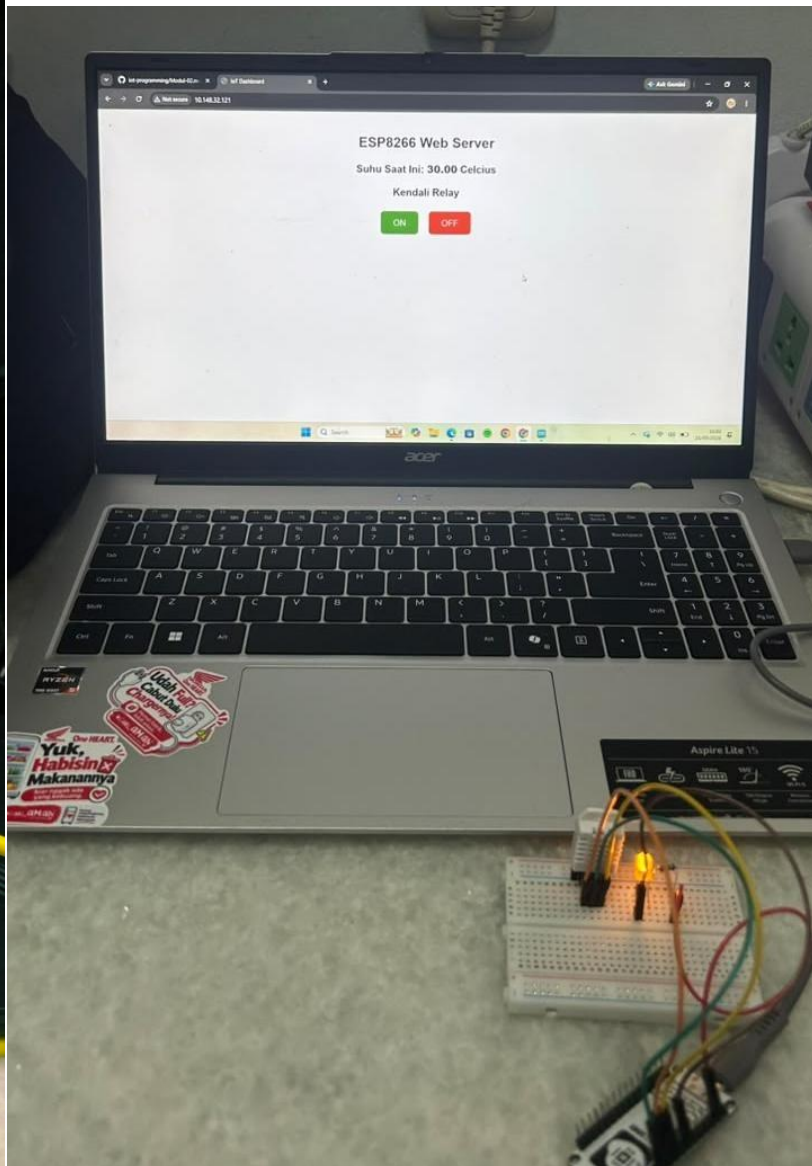
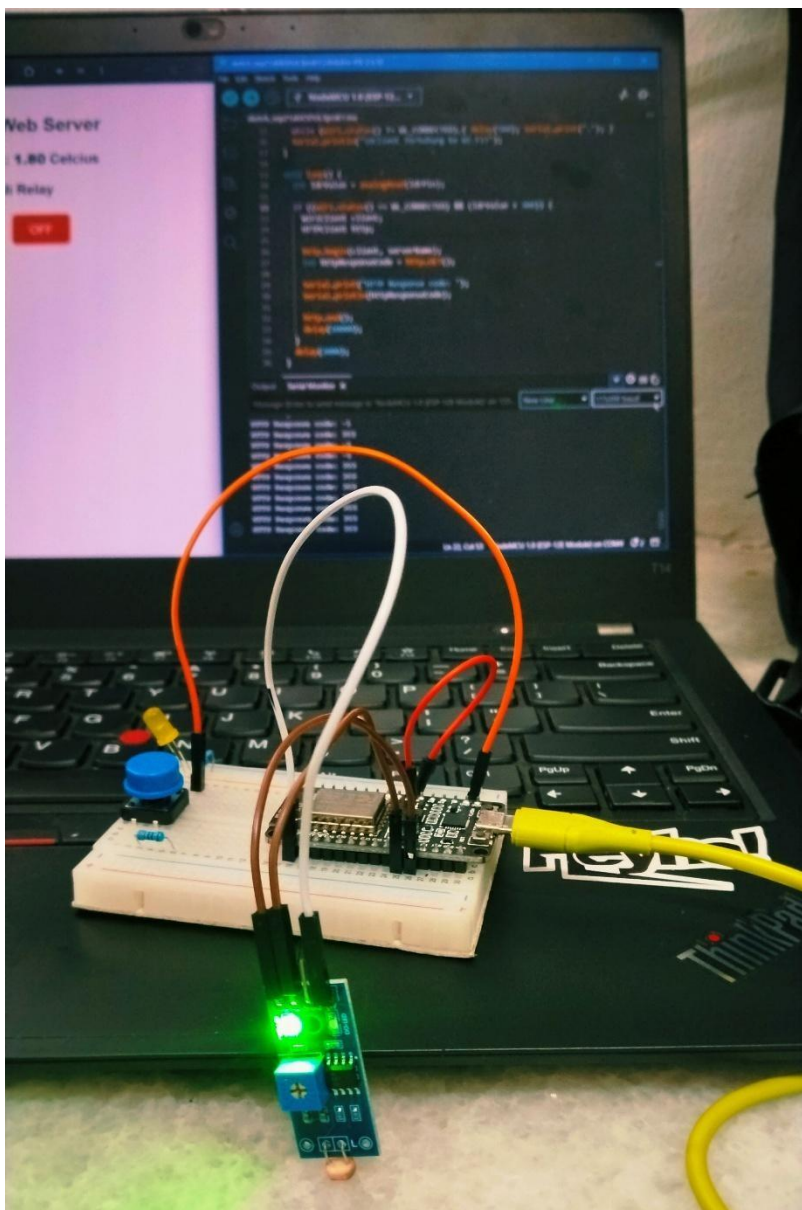
- `ESP8266WebServer server(80);`: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- `html.replace("%TEMPERATURE%", String(t));`: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* `%TEMPERATURE%`, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- `server.on("/relay/on", handleRelayOn);`: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran `/relay/on`, server akan menjalankan aksi mengubah *state* LED (menyalakan/mematikan output), lalu me-*redirect* layar pengunjung kembali ke halaman depan (Location: /) via kode 303.
- `server.handleClient();`: Berada di dalam `loop()` untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

Skenario: Anda sekarang memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi *Client*. Perangkat Klien ini dipasang LDR. Jika LDR tertutup kegelapan, ia akan mengirim sinyal HTTP secara jarak jauh (via jaringan Wi-Fi) untuk menyalakan LED di *Server* milik teman Anda (Praktikum 1).

Langkah Kerja Rangkaian:

1. Rangkai sensor **LDR** (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin **A0**.



Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program di bawah. **Ganti variabel serverName dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda.** (Pastikan Anda menggunakan Wi-Fi yang sama persis).

```
sketch_sep21aMODUL3latihan2.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4
5  const char* ssid = "505";
6  const char* password = "16112006";
7
8  const char* serverName = "http://10.148.32.121/relay/on";
9  const byte ldrPin = A0;
10
11 void setup() {
12     Serial.begin(115200);
13     WiFi.mode(WIFI_STA);
14     WiFi.begin(ssid, password);
15     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16     Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20     int ldrValue = analogRead(ldrPin);
21
22     if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23         WiFiClient client;
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM') New Line 115200 baud

```
.....
Client Terhubung ke Wi-Fi!
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: 303
```

3. **Upload** kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan **LED** di NodeMCU *Server* milik teman Anda pasti akan menyala!

Penjelasan Singkat Kode:

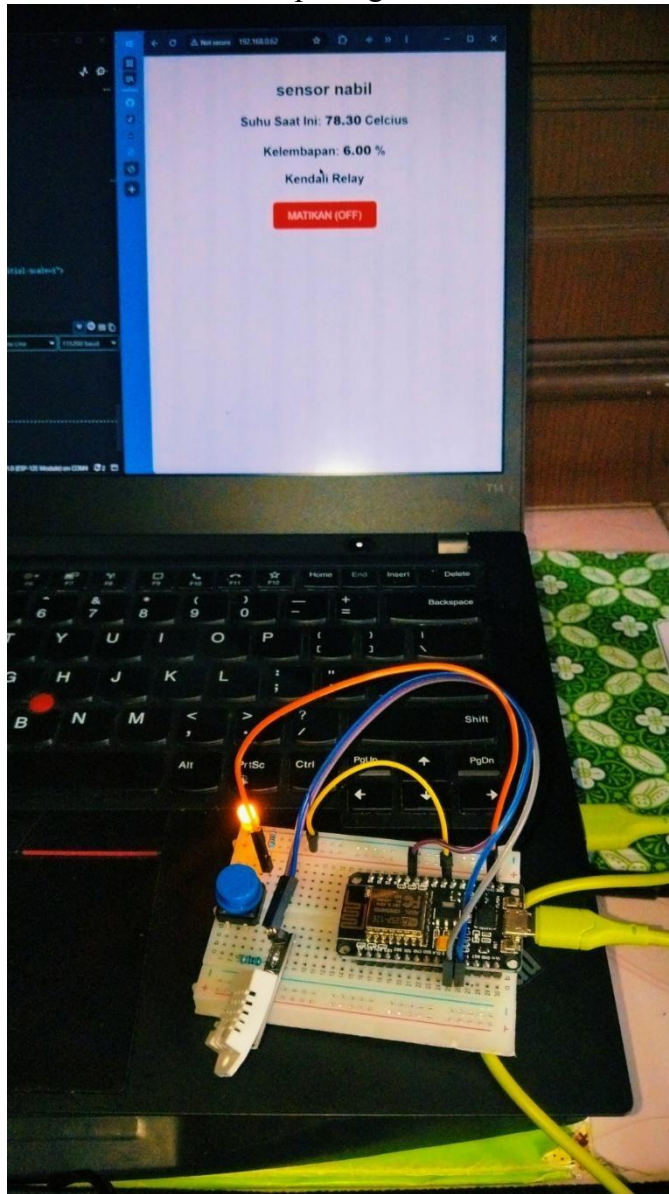
- WiFiClient client; dan HTTPClient http;: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

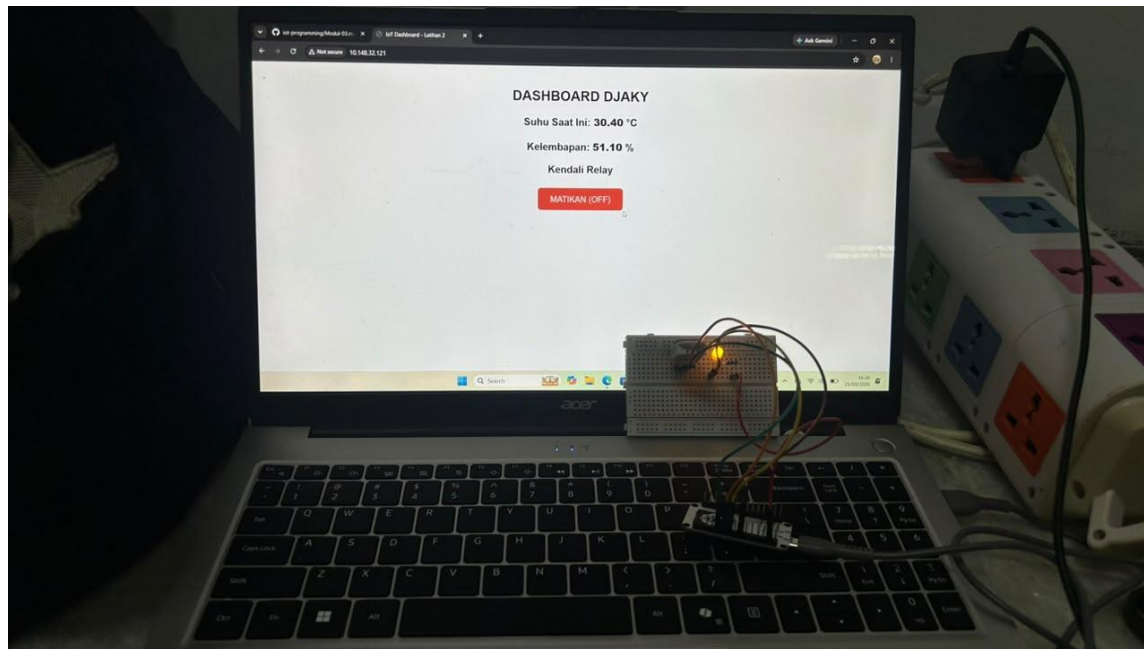
3.6 Latihan

Kerjakan soal berikut secara mandiri untuk menguji penalaran dan modifikasi jaringan.

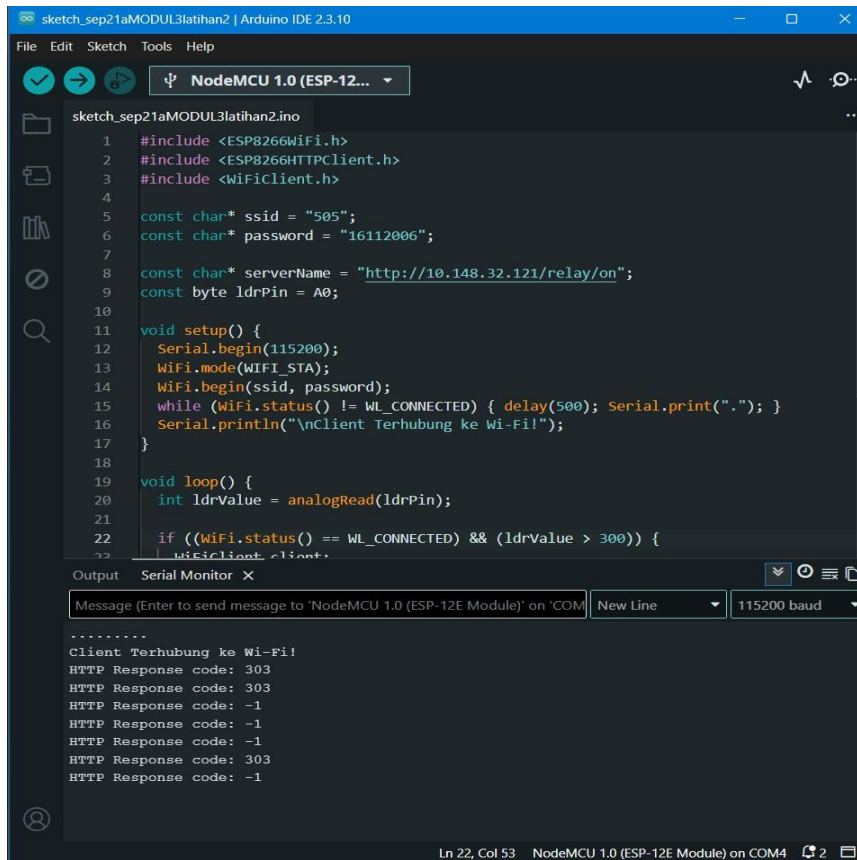
- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?
- **Berada dalam Satu Jaringan / Subnet yang Sama:** Agar NodeMCU yang bertindak sebagai *HTTP Client* (Praktikum 2) dapat mengirimkan *request* ke NodeMCU yang bertindak sebagai *Web Server* (Praktikum 1), kedua perangkat harus terhubung ke *Router*, *Access Point*, atau *Hotspot* eksternal yang sama. Mode *STA* memberikan kedua NodeMCU alamat IP dalam segmen (subnet) jaringan lokal yang sama sehingga keduanya dapat saling berkomunikasi secara langsung.
- **Aksesibilitas Multi-Device Tanpa Mengisolasi Koneksi:** Jika NodeMCU *Server* diatur ke mode *Access Point (AP)*, ia akan memancarkan jaringan Wi-Fi mandiri. Hal ini memaksa perangkat lain (seperti HP, laptop, atau NodeMCU *Client*) untuk memutuskan koneksi Wi-Fi/Internet utama mereka agar bisa terhubung ke Wi-Fi NodeMCU tersebut. Dengan mode **Station (STA)**, semua perangkat (NodeMCU *Server*, NodeMCU *Client*, HP, dan Laptop) dapat tetap terhubung ke satu Wi-Fi utama yang sama tanpa terisolasi dari jaringan luar.
- **Latihan 2 (Modifikasi Antarmuka & Telemetry):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:
 - i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.

- ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:
- Jika LED sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.
 - Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke /relay/off.
- (Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(), periksa status pin relay/LED menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai). Tuliskan potongan baris kode HTML dan fungsi handleRoot() yang Anda ubah!*





- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).



```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "505";
6 const char* password = "16112006";
7
8 const char* serverName = "http://10.148.32.121/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int ldrValue = analogRead(ldrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23     WiFiClient client;
24     HTTPClient http(&client, serverName);
25     http.GET();
26     Serial.print("HTTP Response code: ");
27     Serial.println(http.statusCode());
28   }
29 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') New Line 115200 baud

.....

Client Terhubung ke Wi-Fi!

HTTP Response code: 303

HTTP Response code: 303

HTTP Response code: -1

HTTP Response code: -1

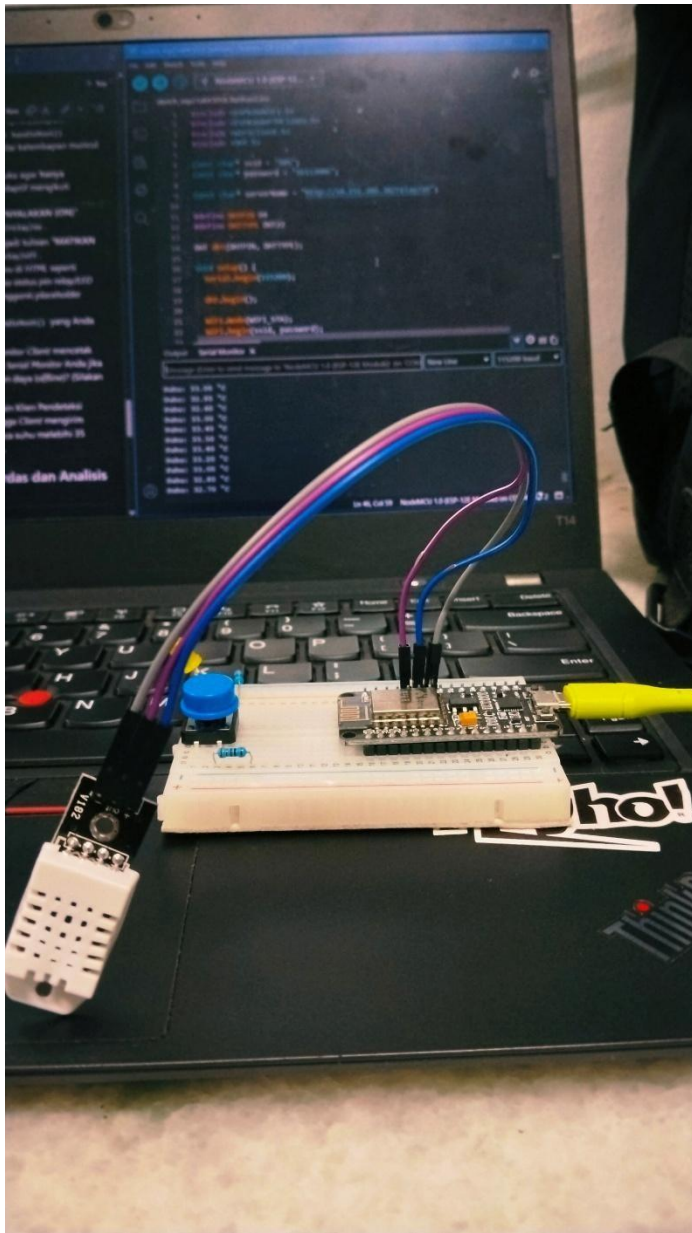
HTTP Response code: -1

HTTP Response code: 303

HTTP Response code: -1

Dia akan menampilkan pesan “HTTP Response code : -1” atau bisa di sebut juga dengan error

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 30 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.



```

sketch_sep21aMODUL3latihan2 | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E...)

sketch_sep21aMODUL3latihan2.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4  #include <DHT.h>
5
6  const char* ssid = "MyRep_85615B_2.4G";
7  const char* password = "5FF456A552";
8
9  const char* serverName = "http://10.151.206.38/relay/on";
10
11 #define DHTPIN D4
12 #define DHTTYPE DHT22
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 void setup() {
17   Serial.begin(115200);
18
19   dht.begin();
20
21   WiFi.mode(WIFI_STA);
22   WiFi.begin(ssid, password);

```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') New L

Client Terhubung ke Wi-Fi!

Suhu: 30.70 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Suhu: 30.60 °C

Ln 6, Col 38 NodeMCU 1.0 (ES

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasinya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (Web Dashboard): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada **Latihan 2**, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- **Tujuan:** Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*). • **Tingkat Kesulitan:** Menengah • **Instruksi:**

- Lakukan modifikasi kode hanya pada sisi **NodeMCU Server (Latihan 2)**.
- Modifikasi variabel Raw Literal `index_html`. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan tag `<head>` untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
- Hint (Petunjuk):* Sintaks standar yang harus disisipkan adalah `<meta http-equiv="refresh" content="5">`.
- Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.



- **Instruksi Analisis Arsitektur:**
 - Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*.
 - Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *Real-Time Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengklik tepat di detik ke-5).
- **Kriteria Keberhasilan:**
 - Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi lingkungan tanpa perlu mengklik tombol *refresh* di *browser*.
 - Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.

